

Reliable 4-Key Press Detection

Design Document & Test Plan

Implementation Language: Standard C | Non-blocking GPIO key scanner

1. Objective

Design a reliable key-press detection mechanism for four active-low GPIO switches. The implementation must reject mechanical switch bounce and short-duration spurious transitions, generate only one event for a continuous key press, avoid blocking operations, and maintain the complete detection state using only one 8-bit variable.

2. Requirements

- Detect four independent GPIO switches.
- Treat the switches as active-low: 0 = pressed, 1 = release.
- Filter mechanical switch bounce and short-duration spurious transitions.
- Generate only one key event while a key remains continuously pressed.
- Allow a new event after the key has been released.
- Do not use blocking delays or wait-for-release loops.
- Use only one 8-bit variable for the complete key-detection state.
- Ignore simultaneous/multiple-key combinations.
- Use portable standard C so the source can be compiled with GCC.

3. Key Mapping

Input Pattern	Key	Meaning
0x0F	NO_KEY	All switches released
0x0E	SW1	Switch 1 pressed
0x0D	SW2	Switch 2 pressed
0x0B	SW3	Switch 3 pressed
0x07	SW4	Switch 4 pressed

4. State Byte Design

The complete state is packed into a single uint8_t variable. The eight bits are divided into three logical fields:

- **Bit 7:** Event flag (1 = event already generated)
- **Bits 6–4:** Debounce counter
- **Bits 3–0:** Current key state

Memory calculation: 1 bit + 3 bits + 4 bits = 8 bits = 1 byte.

Four bits provide 16 possible key patterns, which is sufficient for the five states required (no key plus four single-key states). Three bits provide eight counter values, which is sufficient for a debounce threshold of five samples. One bit is sufficient for the event flag.

5. Debounce Strategy

Mechanical switches can bounce during a press or release, producing rapid transitions that could otherwise be interpreted as multiple key presses. The scanner therefore samples the GPIO periodically and requires the input to remain unchanged for a predefined number of consecutive samples before accepting it as stable.

The configured debounce limit is 5 consecutive stable samples. If the scanner is called every 1 ms, this corresponds to approximately 5 ms of stable input before an event is accepted. No `delay()` or busy-wait loop is used; the scanner performs a small amount of work and returns immediately.

6. Long-Press Handling

After a stable key press is confirmed, the event flag is set. Further scans while the key remains pressed do not generate another event. When a stable NO_KEY state is detected, the event flag is cleared, allowing the next press to generate a new event.

7. Multiple-Key Handling

Only one key at a time is required. Patterns that do not match NO_KEY, SW1, SW2, SW3, or SW4 are treated as invalid/multiple-key input and are ignored.

8. Non-Blocking Architecture

`key_scan()` (or `check_switches()`) is intended to be called periodically, for example from a 1 ms timer tick or scheduler. It does not wait for the switch to be released and does not use a blocking delay.

Typical flow:

1. Read the GPIO input.
2. Detect whether the key state changed.
3. If changed, store the new state and restart the debounce count.
4. If unchanged, increment the debounce count.
5. Once the input has remained stable for the required samples, process a valid key.
6. Set the event flag so a held key does not repeat.
7. Clear the event flag after stable release.

9. Hardware Abstraction

The macro READ_KEYS() represents the GPIO hardware interface. It is intentionally separated from the key-detection algorithm so the code remains portable C. On a real microcontroller, READ_KEYS() can be mapped to the appropriate GPIO input register. The current source uses a fixed placeholder value so that the algorithm remains compliant on a standard GCC environment without target-specific headers or registers.

10. Source Code

C

```
#include <stdint.h>
```

```
/* Switch definitions - Active Low */
```

```
#define NO_KEY 0x0F
```

```
#define SW1 0x0E
```

```
#define SW2 0x0D
```

```
#define SW3 0x0B
```

```
#define SW4 0x07
```

```
/* State byte masks */
```

```
#define KEY_MASK 0x0F
```

```
#define COUNT_MASK 0x70
```

```
#define EVENT_FLAG 0x80
```

```
#define COUNT_SHIFT 4
```

```
#define DEBOUNCE_LIMIT 5
```

```
/* Replace with actual GPIO register read */
```

```
#define READ_KEYS() (NO_KEY)
```

```
static void check_switches(void)
```

```
{
```

```
    /* Bit 7 -> Event flag | Bits 6-4 -> Counter | Bits 3-0 -> Key state */
```

```
    static uint8_t state = NO_KEY | (DEBOUNCE_LIMIT << COUNT_SHIFT);
```

```
    /* Detect change in key input */
```

```

if ((READ_KEYS() & KEY_MASK) != (state & KEY_MASK))
{
    state = (READ_KEYS() & KEY_MASK)
        | (1U << COUNT_SHIFT)
        | (state & EVENT_FLAG);
    return;
}

/* Count consecutive stable samples */
if ((state & COUNT_MASK) < (DEBOUNCE_LIMIT << COUNT_SHIFT))
{
    state += (1U << COUNT_SHIFT);
    return;
}

/* Process the key only after it is stable */
if ((state & EVENT_FLAG) == 0)
{
    switch (state & KEY_MASK)
    {
        case SW1:
            /* Handle SW1 */
            state |= EVENT_FLAG;
            break;

        case SW2:
            /* Handle SW2 */
            state |= EVENT_FLAG;
            break;

        case SW3:

```

```

        /* Handle SW3 */

        state |= EVENT_FLAG;

        break;

    case SW4:

        /* Handle SW4 */

        state |= EVENT_FLAG;

        break;

    default:

        /* No key or multiple keys */

        break;
}
}

/* Reset event flag after key release */
if ((state & KEY_MASK) == NO_KEY)
{
    state &= (uint8_t)~EVENT_FLAG;
}
}

int main(void)
{
    /*
     * check_switches() should be called periodically,
     * for example, every 1 ms from a timer/scheduler.
     */
    while (1)
    {
        check_switches();
    }
}

```

```

}
return 0;
}

```

11. Test Plan

Test Case	Input	Expected Result	Status
No key	0x0F continuously	No key event	To test
SW1 press	0x0E stable for ≥ 5 samples	One SW1 event	To test
SW2 press	0x0D stable for ≥ 5 samples	One SW2 event	To test
SW3 press	0x0B stable for ≥ 5 samples	One SW3 event	To test
SW4 press	0x07 stable for ≥ 5 samples	One SW4 event	To test
Bounce	Alternating key/no-key before stability	No false event	To test
Short noise	Brief invalid transition	Ignored	To test
Long press	One key held for many scans	Only one event	To test
Release	Return to 0x0F	Event flag cleared	To test
Re-press	Release then presses same key	New event generated	To test

Test Case	Input	Expected Result	Status
Multiple keys	Invalid combination	Ignored	To test
Rapid changes	Several different patterns	No false event until stable	To test

12. Build

The source is standard C and can be compiled with GCC using:

13. Limitations and Assumptions

- The GPIO input is assumed to be active-low.
- The scanner must be called periodically for the debounce counter to represent a real time interval.
- A 1 ms scan period is assumed when describing a 5 ms debounce period.
- Simultaneous/multiple-key detection is intentionally not implemented.
- READ_KEYS() is a hardware abstraction placeholder and must be mapped to actual GPIO hardware for deployment.
- The code focuses on key detection; application-specific actions for SW1–SW4 are represented by comments.

14. Conclusion

The proposed design provides non-blocking, debounced detection of four GPIO switches while keeping the complete detection state within one 8-bit variable. The bit-packed state representation, consecutive-sample debounce mechanism, and event flag prevent false and repeated key events while keeping the implementation portable and suitable for integration with a bare-metal embedded system.